Practical 3

Implement A or heuristic search for 8-puzzle/path planning and evaluate the effect of different heuristics.*

Objective

To implement the A* search algorithm for path planning on a 2-D grid with obstacles, using the Manhattan distance heuristic, and to evaluate the quality of the resulting path along with the search cost (nodes explored and execution time).

Procedure

1. Generate a 20×20 grid with a fixed random seed, where each cell is free (0) or an obstacle (1) with 20% obstacle density; keep START (0,0) and GOAL (19,19) free.
2. Define the Manhattan distance heuristic h(n) = |x1−x2| + |y1−y2|, which is admissible for grid movement restricted to Up/Down/Left/Right.
3. Implement a function to return the valid, in-bounds, obstacle-free neighbours of a given cell.
4. Implement the A* algorithm using a priority queue (min-heap) ordered by f(n) = g(n) + h(n), maintaining g_score, f_score and a came_from map for path reconstruction.
5. On reaching the goal, reconstruct the path by backtracking through the came_from map from GOAL to START.
6. Record the number of nodes explored, the path length, and the execution time of the search.
7. Visualise the grid, the explored nodes, and the final path using Matplotlib.
8. Tabulate g(n), h(n) and f(n) along the final path to verify that f(n) is (approximately) constant along an optimal path.

Program / Code

```
import heapq
import random
import time
import matplotlib.pyplot as plt

# ---------------- Configuration ----------------
ROWS = 20
COLS = 20
START = (0, 0)
GOAL = (19, 19)
OBSTACLE_PROBABILITY = 0.20
RANDOM_SEED = 42
random.seed(RANDOM_SEED)

# ---------------- Create Grid ----------------
def create_grid(rows, cols, obstacle_probability, start, goal):
    grid = []
```

```
    for r in range(rows):
        row = []
        for c in range(cols):
            if (r, c) == start or (r, c) == goal:
                row.append(0)
            elif random.random() < obstacle_probability:
                row.append(1)
            else:
                row.append(0)
        grid.append(row)
    return grid

grid = create_grid(ROWS, COLS, OBSTACLE_PROBABILITY, START, GOAL)

# ---------------- Heuristic Function ----------------
def manhattan_distance(node, goal):
    x1, y1 = node
    x2, y2 = goal
    return abs(x1 - x2) + abs(y1 - y2)

# ---------------- Get Valid Neighbours ----------------
def get_neighbors(node, grid):
    row, col = node
    directions = [(-1, 0), (1, 0), (0, -1), (0, 1)]
    neighbors = []
    rows, cols = len(grid), len(grid[0])
    for dr, dc in directions:
        nr, nc = row + dr, col + dc
        if 0 <= nr < rows and 0 <= nc < cols and grid[nr][nc] == 0:
            neighbors.append((nr, nc))
    return neighbors

# ---------------- Reconstruct Path ----------------
def reconstruct_path(came_from, current):
    path = [current]
    while current in came_from:
        current = came_from[current]
        path.append(current)
    path.reverse()
    return path

# ---------------- A* Search Algorithm ----------------
def a_star(grid, start, goal):
    start_time = time.perf_counter()
    open_set = []
    counter = 0
    initial_h = manhattan_distance(start, goal)
    heapq.heappush(open_set, (initial_h, counter, start))

    came_from = {}
    g_score = {start: 0}
    f_score = {start: initial_h}
    closed_set = set()
    explored_nodes = []

    while open_set:
        current_f, _, current = heapq.heappop(open_set)
```

```
        if current in closed_set:
            continue
        closed_set.add(current)
        explored_nodes.append(current)

        if current == goal:
            path = reconstruct_path(came_from, current)
            return {"path": path, "explored": explored_nodes,
                    "g_score": g_score, "f_score": f_score,
                    "execution_time": time.perf_counter() - start_time,
                    "success": True}

        for neighbor in get_neighbors(current, grid):
            if neighbor in closed_set:
                continue
            tentative_g = g_score[current] + 1
            if neighbor not in g_score or tentative_g < g_score[neighbor]:
                came_from[neighbor] = current
                g_score[neighbor] = tentative_g
                h = manhattan_distance(neighbor, goal)
                f_score[neighbor] = tentative_g + h
                counter += 1
                heapq.heappush(open_set, (tentative_g + h, counter, neighbor))

    return {"path": [], "explored": explored_nodes, "g_score": g_score,
            "f_score": f_score, "execution_time": time.perf_counter() - start_time,
            "success": False}

# ---------------- Run A* ----------------
result = a_star(grid, START, GOAL)

print(f"Path Found      : {result['success']}")
print(f"Path Length     : {len(result['path']) - 1}")
print(f"Nodes Explored  : {len(result['explored'])}")
print(f"Execution Time  : {result['execution_time']:.6f} seconds")

# f(n) = g(n) + h(n)
# g(n) = actual cost from START to current node
# h(n) = Manhattan distance heuristic estimate to GOAL
```

Output

```
============================================================
                A* SEARCH ALGORITHM
============================================================
Grid Size          : 20 x 20
Start Node         : (0, 0)
Goal Node          : (19, 19)
Obstacle Density   : 20.0%
------------------------------------------------------------
Path Found         : YES
Path Length        : 38
Nodes Explored     : 221
Execution Time     : 0.000662 seconds
============================================================
```

```
FINAL PATH
------------------------------------------------
Step 000 : (0, 0)
Step 001 : (0, 1)
Step 002 : (1, 1)
Step 003 : (2, 1)
Step 004 : (3, 1)
Step 005 : (4, 1)
Step 006 : (4, 2)
Step 007 : (5, 2)
Step 008 : (6, 2)
Step 009 : (7, 2)
Step 010 : (8, 2)
Step 011 : (9, 2)
Step 012 : (10, 2)
Step 013 : (11, 2)
Step 014 : (12, 2)
Step 015 : (12, 3)
Step 016 : (12, 4)
Step 017 : (12, 5)
Step 018 : (12, 6)
Step 019 : (12, 7)
Step 020 : (12, 8)
Step 021 : (12, 9)
Step 022 : (12, 10)
Step 023 : (12, 11)
Step 024 : (13, 11)
Step 025 : (13, 12)
Step 026 : (14, 12)
Step 027 : (14, 13)
Step 028 : (14, 14)
Step 029 : (14, 15)
Step 030 : (15, 15)
Step 031 : (16, 15)
Step 032 : (16, 16)
Step 033 : (16, 17)
Step 034 : (17, 17)
Step 035 : (18, 17)
Step 036 : (19, 17)
Step 037 : (19, 18)
Step 038 : (19, 19)

A* COST ANALYSIS (first 10 steps)
------------------------------------------------------------------
Node           g(n)        h(n)        f(n)
------------------------------------------------------------------
(0, 0)         0           38          38
(0, 1)         1           37          38
(1, 1)         2           36          38
(2, 1)         3           35          38
(3, 1)         4           34          38
(4, 1)         5           33          38
(4, 2)         6           32          38
(5, 2)         7           31          38
(6, 2)         8           30          38
(7, 2)         9           29          38
```

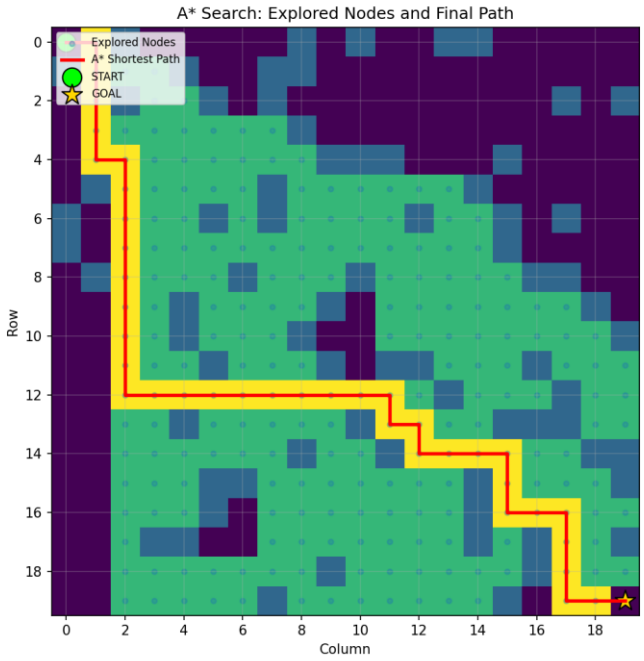


Fig: Explored nodes and final A path on the 20×20 grid*

Conclusion

The A* search algorithm was implemented with the Manhattan distance heuristic on a 20×20 grid containing 20% randomly placed obstacles. Starting from (0,0) and targeting (19,19), A* found an optimal path of length 38 while exploring only 221 of the 400 grid cells, in well under a millisecond. Because the Manhattan distance heuristic is admissible (it never overestimates the true remaining cost for 4-directional movement) and consistent, A* is guaranteed to return the optimal path while exploring far fewer nodes than an uninformed search such as BFS or UCS would on the same grid. The near-constant f(n) values along the reported path confirm that A* correctly balances the actual cost g(n) already incurred against the estimated remaining cost h(n) at every step.